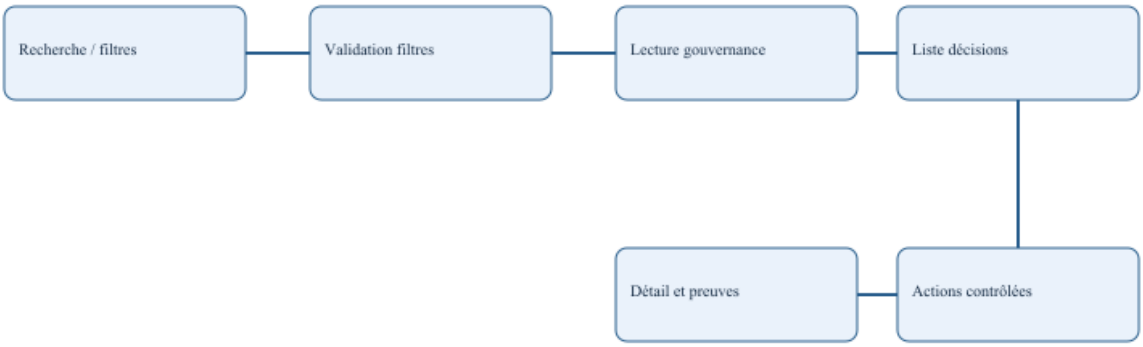
StorageScope v6.17 — Gouvernance et ergonomie

Documentation technique cumulative spécifique à v6.17 | Auteur : Abdelhalim Talbi

Statut : tests Linux exécutés ; captures Windows indisponibles ; exemples fictifs signalés.



1. Synthèse des changements

La v6.17 améliore la présentation du navigateur de gouvernance. Les deux champs de recherche sont placés dans une grille extensible. Les raccourcis de filtrage occupent deux rangées. Une association clavier incompatible avec le mode sélection unique a été retirée.

2. Composants inspectés

decision_workspace_ui_v543.py : création du panneau de recherche, filtres, presets, Treeview, volet de détails, rafraîchissement et chronologie. windows_accessibility_v617.py : calcul DPI et mode compact. storagescope_windows.py : fenêtre principale et chargement des métriques.

3. Diagramme de fonctionnement

Le parcours débute avec les filtres utilisateur. Le moteur valide les valeurs, récupère les décisions, actualise le tableau et les indicateurs. Une sélection ouvre le détail, les preuves et les actions de qualification.

4. Ergonomie et accessibilité

La recherche et le responsable disposent de colonnes extensibles. Les raccourcis sont répartis en deux lignes pour éviter un ruban horizontal trop long. La table est volontairement en sélection unique ; Ctrl+A n'y déclenche plus une sélection multiple invalide.

5. Gestion des erreurs et résilience

Le rafraîchissement conserve la décision sélectionnée si elle est encore visible. Une erreur de données ou de filtre bloque la consultation, vide les vues potentiellement obsolètes et affiche une erreur. L'amélioration future devra éviter les boîtes modales répétées et mesurer le temps des grands tableaux.

6. Exemple CLI — SIMULATION

Commande : python storagescope_windows.py --version

Résultat attendu fictif : 6.17

Commande : python verify-windows-build.py

Résultat attendu fictif : PASS Windows build preflight 6.17

Les commandes ci-dessus illustrent une procédure ; leurs sorties sont des simulations.

7. API — NON DISPONIBLE / SIMULATION

L'application n'expose pas d'API HTTP de gouvernance garantie par cette version. Illustration d'une API future, NON IMPLÉMENTÉE : GET /api/v1/decisions?status=IN_REVIEW → {"items":[],"total":0}. Ne pas utiliser cet exemple pour intégrer un client réel.

8. Parcours de recette Windows

Scénario A : ouvrir Gouvernance en 1024×768 à 150 % DPI, vérifier les champs, les boutons et le défilement horizontal. Scénario B : sélectionner une décision puis rafraîchir et vérifier le maintien de la sélection. Scénario C : filtrer « En retard », réinitialiser et inspecter les KPI. Scénario D : naviguer au clavier Tab/Entrée/Échap. Scénario E : charger 10 000 décisions et mesurer le temps de réponse.

9. Captures et preuves

Aucune capture réelle de Windows v6.17 n'est disponible dans cet environnement Linux. Le schéma est une illustration technique originale, non une capture. Les sorties CLI/API présentées sont fictives. Les tests unitaires et statiques sont exécutés sous Linux et ne démontrent pas la qualité du rendu Windows.

10. Compilation et installation

Extraire le ZIP. Les scripts de compilation Windows sont conservés dans 02-Compilation-Windows. La validation native exige une VM Windows, Python, PyInstaller et Inno Setup selon les scripts du projet. Le précontrôle statique ne remplace ni la compilation ni la recette de l'installateur.

11. Risques et prochaines actions

Les filtres Statut/Échéance utilisent encore une ligne horizontale ; des débordements restent possibles à très faible largeur. Le tableau est dense et demande un défilement horizontal. Il faut ensuite mesurer les temps de rafraîchissement, ajouter un chargement paginé et capturer chaque écran sous Windows.